FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY DEPARTMENT OF COMMUNICATION TECHNOLOGY AND NETWORK

GROUP PROJECT

DESIGN AND ANALYSIS ALGORITHMS

(CSC4202) SEMESTER II 2025/2026

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSC4202-1

GROUP: 10

PROJECT TITLE: AMBULANCE ROUTING SYSTEM IN UPM

GITHUB REPOSITORY (LINK):

https://github.com/Fushi23/CSC4202-G10-Ambulance-Routing-System/tree/main

---

LECTURER: DR NUR ARZILAWATI MD YUNUS

NAME		MATRIX NUMBER
1.	MUHAMMAD IRFAN DANIEL BIN MD SHAM	226089
2.	MUHAMMAD AIDIL QAYYIM BIN DIN	223555
3.	ZAHIN ZULQARNAIN BIN ZULKIFLI	222292
4.	MUHAMMAD AIMAN HAZIQ BIN MOHD SALLEH	222707

TABLE OF CONTENTS

1.0 Introduction

Emergency response systems require fast and accurate decision-making to minimise response time and improve the chances of saving lives. Determining the shortest and most efficient route for emergency vehicles is one of the most critical challenges in emergency management, especially when road conditions and traffic congestion can heavily affect travel time.In time-sensitive situations, finding the optimal route manually is often impractical because there are just simply so many factors that have to be considered even for someone who has lived at certain area for years.

Graph algorithms provide an effective solution to this problem by modelling road networks as weighted graphs, where locations are represented as vertices and roads are represented as edges with associated travel costs. Dijkstra's Algorithm is widely recognised for its ability to determine the shortest path from a single source to all other reachable vertices in graphs with non-negative edge weights, among the various shortest-path algorithms . Its efficiency and guaranteed optimality make it suitable for real-world routing applications.

In this project, an Ambulance Routing System is developed for the Universiti Putra Malaysia (UPM). The campus road network is modelled as a weighted undirected graph, where edge weights represent the effective travel cost based on both road distance and traffic conditions. The goal of this project is during an emergency scenario involving a fire incident and traffic congestion, determine the most efficient route for an ambulance travelling from Hospital Sultan Abdul Aziz Shah (HSAAS) to KOSASS B. The correctness, efficiency, and complexity of Dijkstra's Algorithm through implementation, testing, and theoretical analysis will also be taken into consideration while evaluating.

2.0 Problem Statement and Scenario

2.1 Geographical Setting

Universiti Putra Malaysia (UPM) is a public research university and is one of the largest universities in Malaysia, which is located in Serdang, Selangor. The campus occupies a large area consisting of various buildings that are scattered in several zones, such as the residential college zone, faculties, recreation centres, and the medical zones. Due to the large size of the campus, road connectivity between buildings becomes essential for movement and also for emergency response.

In this case study, the geographical environment is based on several selected buildings in UPM's campus. The selected buildings are treated as nodes in a weighted undirected graph. In Figure 2.1, the edges between any two nodes represent the roads connecting the buildings, and the weights on the edges represent the estimated distance in kilometers between the buildings. The selected nodes are: **Hospital Sultan Abdul Aziz Shah** (HSAAS), **KOSASS A**, **KOSASS B**, **Kolej Pendeta Za'ba** (KPZ), **Kolej Tun Dr Ismail** (KTDI), **MARDI** (which borders the UPM campus), **Padang Kawad**, **Bukit Ekspo**, **Serumpun**, and **Pusat Sukan**. Figure 2.1 shows the connection of roads between these buildings and their distance. HSAAS is the source node while KOSASS B is the destination node.

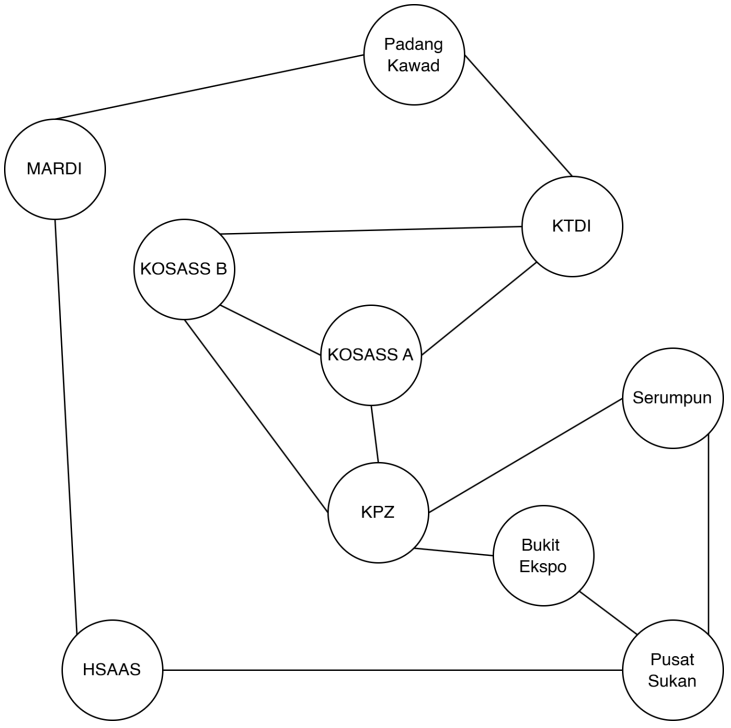


Figure 2.1 Campus road network showing key locations involved in the disaster scenario.

2.2 Disaster Scenario

A fire incident takes place in KOSASS B due to an electrical fault in one of its residential buildings at 1:00 PM on a weekday. This period is considered a peak hour because many students and staff are actively moving around the campus for classes, lunch breaks, and other activities.

At the same time, a football tournament is being held at the field in front of KTDI. The event attracts a large number of participants, spectators, and event staff. As a result, the roads leading to and from KTDI become heavily congested due to increased vehicle traffic and pedestrian movement around the area.

An ambulance from HSAAS must be dispatched to KOSASS B as quickly as possible. However, there are several possible routes between HSAAS and KOSASS B, and some of these routes may be affected by the congestion around KTDI or by emergency response activities near the fire location. Consequently, selecting the fastest route manually may be difficult and time-consuming during an emergency.

2.3 Damage and Impact

Several impacts occur as a result of the fire that occurs in KOSASS B on the campus. There is damage to the structure and electrical systems in the affected residential block, while there can also be damage due to smoke in other neighbouring rooms. In addition, students in the affected block should be evacuated because it is not safe for them to stay in the building.

Another impact of the fire is that of injuries. Some students get smoke inhalation injuries that should be treated. There will be other negative impacts on the health of the students as a result of the delayed arrival of medical aid. Besides the health impacts of the fire, one should note the psychological impacts on the students.

As a result of the fire, the use of roads in certain parts of the campus is impacted. The fire happens along with the occurrence of the football tournament on the campus. This means that there are some routes between HSAAS and KOSASS B that cannot be used properly because of the fire and the marathon.

2.4 Importance of Analysis of Algorithm Design (AAD)

The time taken in such an emergency scenario is crucial, and any additional minutes it would take the ambulance to get to the scene may result in a dire outcome for the wounded students. Estimating the shortest path by guessing or by trial and error for all routes may not be the most effective solution, given the rapidly changing road conditions due to both the fire and the marathon race.

Here comes the significance of AAD, whereby creating a graph from the road map of UPM campus and solving the shortest path problem using appropriate algorithms, it can be determined which route is the most optimal for travelling from HSAAS to KOSASS B. Dijkstra's algorithm has been applied in emergency vehicle systems to automatically route vehicles to their destinations from a given source, giving the shortest path in such road network scenarios.

An algorithm also helps the system take into consideration some practical issues, like congestion weight. Instead of just considering the shortest distance between the points, the algorithm is able to incorporate factors which make the journey longer, hence becoming more practical. Algorithms used in finding the shortest path for emergency cases are not allowed to solely depend on distance due to the changes in traffic situations at different times.

2.5 Goal and Expected Output

The objective of this assignment is to determine the optimal route to be taken by the ambulance from HSAAS to KOSASS B. In this particular case, the weights on the edges of the graph will represent distance in kilometers. The roads which face congestion issues, like roads leading to and out of KTDI because of the football tournament, have been assigned an increased cost to discourage the selection of this route.

The expected output of the algorithm is to present the shortest path from HSAAS to KOSASS B, as well as the distance covered along this path.

3.0 Development of A Model For The Chosen Scenario

3.1 Data Type

The problem needs to be modelled as a weighted undirected graph. In the graph:

- **Nodes (Vertices)** represent physical locations on the UPM campus. There are 10 nodes in total: HSAAS, KOSASS A, KOSASS B, KPZ, KTDI, MARDI, Padang Kawad, Bukit Ekspo, Serumpun, and Pusat Sukan.
- **Edges** represent roads or paths connecting two locations.
- **Weights** represent the effective travel cost between two connected locations. The cost is calculated by multiplying the road distance (in kilometres) by a traffic multiplier that reflects the severity of traffic congestion.

Table 3.1 Traffic multipliers based on the Traffic Level.

Traffic Level	Multiplier
Low	x1.0
Moderate	x1.5
Heavy	x2.0

The effective travel cost is calculated using the following equation:

$$Effective\ Travel\ Cost\ =\ Distance\,(km)\ \times\ Traffic\ Level\ Multiplier$$

For example, the road connecting MARDI and Padang Kawad has a distance of 1.4 km and experiences heavy traffic conditions. Therefore, its effective travel cost is:

$$1.4\ \times\ 2.0\ =\ 2.8$$

Using effective travel cost instead of distance alone allows the routing system to consider both road length and traffic congestion when determining the optimal ambulance route.

3.2 Objective Function

The objective is to minimise the total travel distance from the source node (HSAAS) to the destination node (KOSASS B).

Formally, let $G = (V, E, W)$ be a weighted graph where V is the set of vertices, E is the set of edges, and $w(u, v)$ is the weight of the edge between vertex u and vertex v. The goal is to find a path P = {v₀, v₁, v₂, ..., v□} $P = \{v_0, v_1, v_2, \ldots, v_k$ where $v_0 = HSAAS$ and $v_k = KOSASS\ B$, such that the total path cost is minimised:

Mnimise: $\Sigma w(v_i, v_{i+1})$ for $i = 0$ to $k - 1$

3.3 Constraints

Graph constraints:

- All edge weights are non-negative ($w(u, v) \geq 0$ for all edges).
- The graph is undirected and connected, meaning there is at least one path between any two nodes.
- The ambulance must travel only along existing road edges. It cannot take shortcuts outside the defined graph.

Scenario constraints:

- The peak hour constraints resulted in the MARDI — Padang Kawad corridor and HSAAS — Pusat Sukan corridor being congested.
- The Padang Kawad — KTDI, KTDI — KOSASS A and KTDI — KOSASS B corridor is affected by the football tournament, causing higher effective travel cost along that segment.

Algorithm constraints:

- The algorithm must guarantee the optimal (shortest) path, not just an approximate one.
- The algorithm must be able to handle a single-source to single-destination routing problem.

3.4 Example Dataset

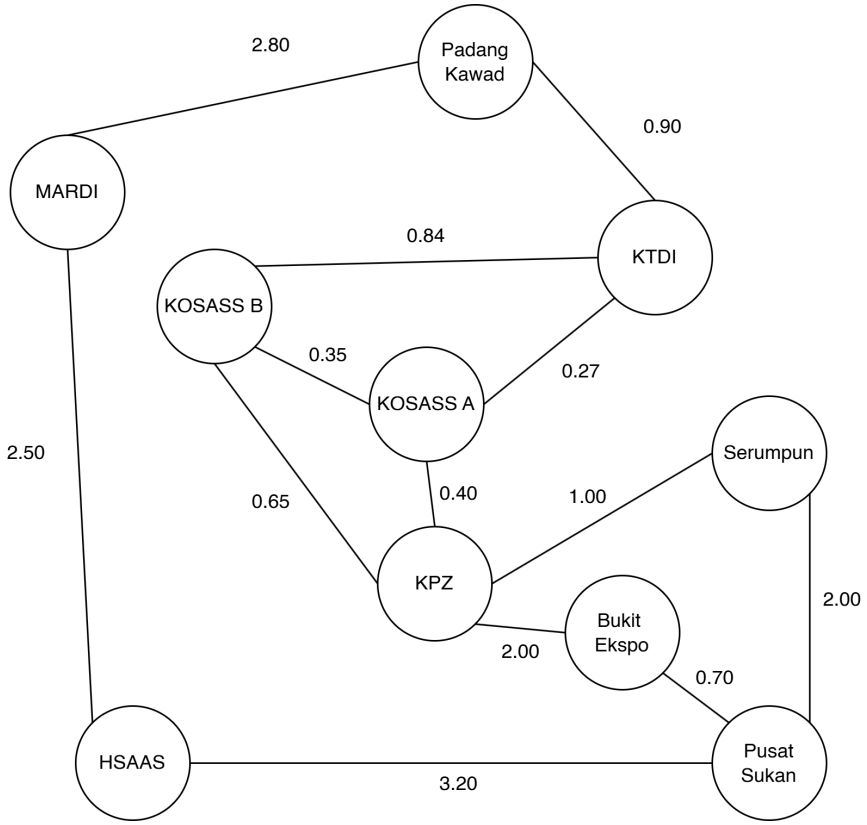


Figure 3.1 Weighted graph of UPM campus road network

The complete edge list extracted from the graph is shown in Table 3.1 below:

Table 3.2 Edge list of the UPM campus road network graph

Edge	**Distance (km)**	**Traffic Level**	**Effective Travel Cost = Distance × Multiplier of Traffic Level**
HSAAS — Pusat Sukan	3.20	Low	3.20
HSAAS — MARDI	2.50	Low	2.50
Pusat Sukan — Serumpun	2.00	Low	2.00
Pusat Sukan — Bukit Ekspo	0.70	Low	0.70
Serumpun — KPZ	1.00	Low	1.00
Bukit Ekspo — KPZ	2.00	Low	2.00
KPZ — KOSASS A	0.40	Low	0.40
KPZ — KOSASS B	0.65	Low	0.65
KOSASS A — KTDI	0.18	Moderate	0.27

KOSASS A — KOSASS B	0.35	Low	0.35
KOSASS B — KTDI	0.56	Moderate	0.84
KTDI — Padang Kawad	0.60	Moderate	0.90
MARDI — Padang Kawad	1.40	Heavy	2.80

Source node: HSAAS
Destination node: KOSASS B

3.5 Additional Requirements (Space, Time, and Value Constraints)

Time constraint: In an emergency, the algorithm must compute the result quickly. The chosen algorithm should be efficient enough to run in a very short time, even if the graph grows larger. Abiy et al. (n.d.) stated that in Dijkstra's algorithm, edges with heavier weights represent more congested or difficult roads, and the algorithm will naturally try to avoid these in favour of lower-cost paths.

Space constraint: The graph in this scenario is relatively small, with 10 nodes and 13 edges. The memory used to store the adjacency list and the priority queue is manageable. The space complexity of the algorithm is $O(V + E)$, where V is the number of vertices and E is the number of edges.

Value constraint: Only non-negative edge weights are allowed in this model. This is because the algorithm selected (Dijkstra's) requires all weights to be non-negative to guarantee correctness. Dijkstra's algorithm can only work with graphs that have positive weights. If a negative weight exists in the graph, the algorithm will not produce a correct result (Hernandez, 2020). In the context of this scenario, all road distances are physical distances and are always positive values, so this constraint is naturally satisfied.

4.0 Specification of an Algorithm

4.1 Selected Topic and Algorithm

The topic selected for this project is Graph Algorithms, specifically the Single-Source Shortest Path problem. The algorithm chosen to solve this problem is Dijkstra's Algorithm.

Dijkstra's algorithm is a greedy algorithm designed to find the shortest paths from a single source node in a weighted graph with non-negative edge weights. Chauhan (2024) stated that tt was proposed by Edsger W. Dijkstra in 1956 and remains one of the most widely used algorithms in computer science.

The algorithm works by starting from the source node and progressively exploring the nearest unvisited nodes. At every step, it picks the node with the smallest known distance from the source and treats that distance as permanent. It maintains a record of the shortest known distance to every node and updates these distances whenever it finds a shorter path through a neighbour. Once the algorithm determines the shortest path between the source node and another node, that node is marked as visited and added to the path. This continues until all nodes in the graph have been processed.

In the context of this scenario, the source node is HSAAS (node A) and the destination node is KOSASS B. The algorithm will evaluate all possible paths and return the one with the lowest total effective travel cost.

4.2 Reason for Selection and Comparison with Other Algorithms

Before deciding on Dijkstra's algorithm, three other algorithms were evaluated as potential solutions. These are **Breadth-First Search (BFS)**, **Floyd-Warshall**, and **Greedy Best-First Search**. The comparison is summarised in Table 4.1 at the end of this section.

4.2.1 Breadth-First Search (BFS)

BFS is one of the most fundamental graph traversal algorithms. It explores a graph level by level, starting from the source node and visiting all neighbouring nodes before moving further out. For unweighted graphs, or graphs where all edges have the same weight, BFS will always reach the destination node by traversing the least number of edges. This means

the first time BFS reaches the destination during traversal, the nodes visited along the way represent the shortest path.

However, BFS does not consider edge weights. It treats every edge as having a uniform cost of 1 regardless of the actual weight assigned to it. In a weighted graph, the first time BFS discovers a node does not guarantee the shortest path to that node. BFS has no way of knowing whether a particular discovery of a node gives the shortest path, so the only way it could find the shortest path in a weighted graph would be to search the entire graph and keep recording the minimum distance, which is not a practical solution for larger networks (Malhotra, 2018).

In the UPM campus graph, the edge weights represent effective travel costs that differ across each road. For weighted graphs, BFS does not minimise cost but instead minimises the number of hops, which leads to incorrect answers in weighted scenarios. Dijkstra's algorithm is designed specifically for weighted graphs with non-negative edge weights. Because of this, BFS is not suitable for this problem.

4.2.2 Floyd-Warshall Algorithm

The Floyd-Warshall algorithm finds shortest paths in a directed weighted graph with positive or negative edge weights. A single execution of the algorithm finds the shortest path lengths between all pairs of vertices. This makes it an all-pairs shortest path algorithm, meaning it computes the shortest distances between every possible combination of nodes, not just from one source.

For this project, computing paths between all pairs of nodes is completely unnecessary. Only one specific route is needed, which is from HSAAS to KOSASS B. Floyd-Warshall is the slowest among common shortest path algorithms and wastes space when applied to problems with many different types of points (Wang et al., 2018). Its time complexity is $O(V^3)$, which is significantly higher than Dijkstra's $O((V + E) \log V)$. For a graph with 10 nodes and 13 edges like this one, Floyd-Warshall would perform far more unnecessary computations. It is a better fit for situations where you need the distances between every pair of nodes, not a single-source emergency routing problem.

4.2.3 Greedy Best-First Search

Greedy Best-First Search is a search algorithm that always expands the node that looks closest to the destination based on a heuristic function. Unlike Dijkstra's, it does not track the actual cumulative cost from the source. It only considers the estimated cost left to reach the

goal. Greedy Best-First Search is not guaranteed to find the shortest path. It runs faster than Dijkstra's because the heuristic guides it towards the goal quickly, but the trade-off is that it can end up choosing a path that is clearly not the shortest when there are obstacles or complex routes involved (Patel, n.d.).

The main problem here is that this algorithm does not guarantee an optimal result. Greedy Best-First Search does not guarantee the shortest or most optimal solution because it only focuses on the most promising path at each step and can get trapped in local optima. In an emergency scenario like this one, finding the actual shortest path is critical. A suboptimal route could cost valuable minutes and worsen the condition of injured students. Additionally, Greedy Best-First Search still requires a heuristic function based on node coordinates, which are not available in this graph model. So, even setting aside the optimality issue, the algorithm cannot be properly applied here.

4.2.4 Why Dijkstra's Algorithm is Chosen

Dijkstra's algorithm is the most appropriate choice for this problem for several reasons. First, all edge weights in the UPM campus graph are non-negative effective travel costs, which satisfies the core requirement of Dijkstra's algorithm. Second, unlike BFS, Dijkstra's fully accounts for edge weights and always finds the path with the lowest total cost rather than the fewest hops. Third, unlike Floyd-Warshall, it is focused only on the single-source problem and does not compute unnecessary paths between all pairs of nodes. And unlike Greedy Best-First Search, it guarantees an optimal result because it tracks the true cumulative cost from the source at every step.

Dijkstra's algorithm works because of one core guarantee: in a graph with non-negative edge weights, the node with the smallest tentative distance is permanently settled the first time it is removed from the priority queue. Because all weights are non-negative, no future path through any unsettled node can produce a shorter route (Code Intuition, n.d.).

In emergency response contexts, Dijkstra's algorithm has been widely applied for ambulance routing because it reliably finds the shortest path in road networks where all travel costs are positive. For this scenario involving 10 nodes and 13 edges with a clearly defined source and destination, Dijkstra's algorithm provides the right balance of simplicity, correctness, and efficiency.

Table 4.1: Comparison of shortest path algorithms

Algorithm	Type	Consider Edge Weight	Time Complexity	Guarantees Optimal Path	Suitable
Dijkstra's	Single-source shortest path	Yes	O((V+E) log V)	Yes	Yes
BFS	Graph traversal	No	O(V+E)	No (ignores weights)	No
Floyd-Warshall	All-pairs shortest path	Yes	$O(V^3)$	Yes	No (computes all pairs, too slow)
Greedy Best-First	Heuristic search	No (Heuristic Only)	O(b^d) worst case	No	No (suboptimal, needs coordinates)

5.0 Implementation

This section explains how Dijkstra's algorithm is implemented to solve the ambulance routing problem that was described before. To make the explanation and the code easier to read, each location on the campus graph is renamed using a single letter. Table 5.1 shows the mapping between the letters and the real location names. After the mapping, this section presents the idea of the algorithm, the flowchart, the pseudocode, the Java code and the result from the program testing.

5.1 Dijkstra's Algorithm

Dijkstra's algorithm is a method that finds the shortest path from one starting node to the other nodes inside a weighted graph. It works in a greedy way because at every step it picks the unvisited node that has the smallest known distance from the source and treats that distance as final (Cormen, 2018). In our problem the source node is A (HSAAS) and the destination node is J (KOSASS B).

The algorithm keeps a distance value for every node. At the beginning the distance of the source is set to 0 and all the other nodes are set to infinity because they have not been reached yet (MIT OpenCourseWare, 2020). After that the algorithm keeps doing a step called relaxation. Relaxation means it checks whether the path to a neighbour through the current node is shorter than the distance that was stored before, and if it is shorter the neighbour distance is updated to the smaller value (MIT OpenCourseWare, 2020).

To always take the nearest node fast, the algorithm uses a priority queue. A priority queue based on a min-heap always returns the item with the lowest value first, so it is suitable to get the closest unvisited node in each round (Code and Debug, 2025). Using a priority queue also makes the algorithm faster than checking every single node one by one in each step (MIT OpenCourseWare, 2020). One more thing to note is that Dijkstra's algorithm only gives a correct answer when all the edge weights are not negative (Hernandez, 2020). In our scenario this is fine because all the effective travel costs are positive numbers.

Table 5.1 Mapping of each campus location to a letter

Letter	Location Name	Role
A	Hospital Sultan Abdul Aziz Shah (HSAAS)	Source
B	MARDI	-
C	Pusat Sukan	-
D	Padang Kawad	-
E	Serumpun	-
F	Bukit Ekspo	-
G	Kolej Tun Dr. Ismail (KTDI)	-
H	Kolej Pendeta Za'ba (KPZ)	-
I	KOSASS A	-
J	KOSASS B	Destination

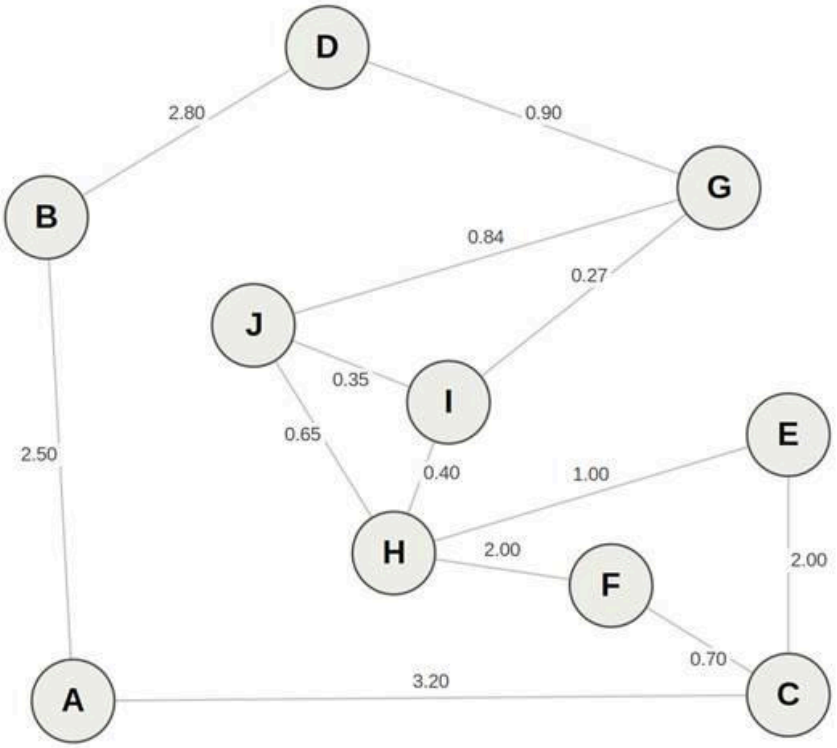


Figure 5.1 The UPM campus graph with every location shown as a letter

5.2 Flowchart

Figure 5.2 shows the overall flow of the algorithm that we use for the ambulance routing. The process begins by setting the distance of node A to 0 and the rest of the nodes to infinity. After that the program keeps taking the nearest node from the priority queue. If that node was already visited before, the program just skips it. If it was not visited, the program relaxes all of its neighbours and updates their distances when a shorter path is found. This loop keeps running until the priority queue becomes empty. In the end the program rebuilds the path from J back to A and prints out the result.

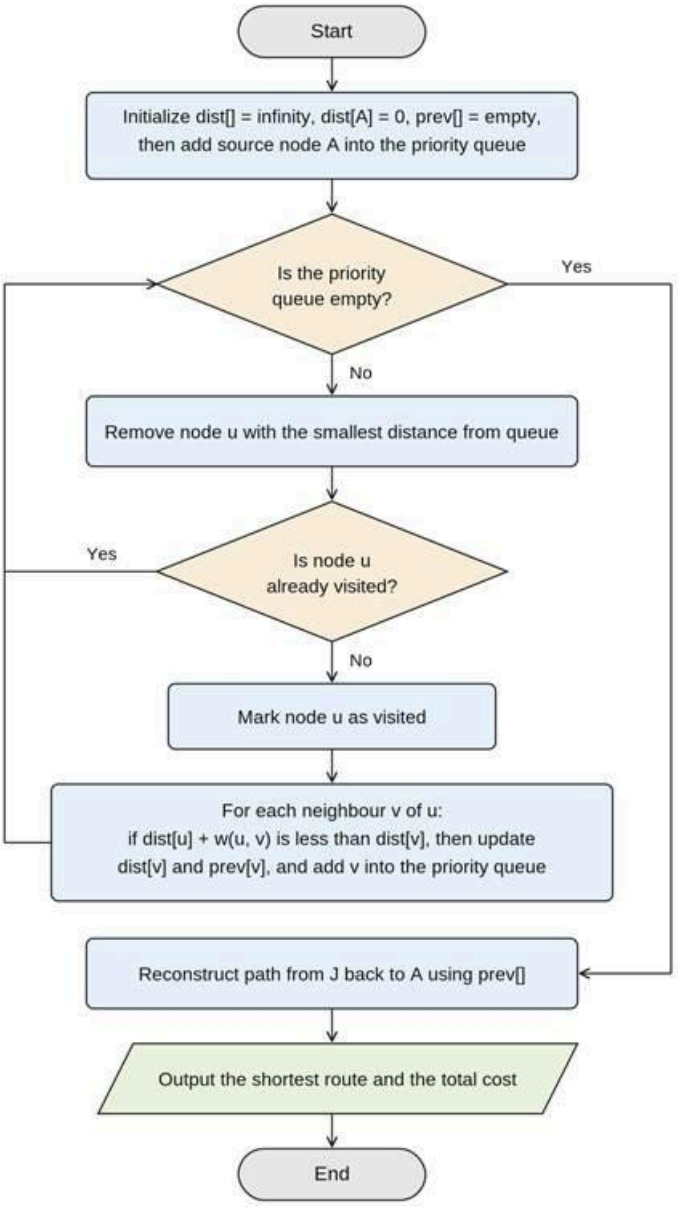


Figure 5.2 Flowchart of Dijkstra's algorithm for the ambulance routing problem

5.3 Pseudocode

The pseudocode below describes the steps of the algorithm in a simple form before it is turned into real Java code. The part that repeats the most is the while loop and the relaxation step inside it, because this is where the shortest distances keep getting improved until no shorter path can be found.

```
ALGORITHM Dijkstra(Graph, source, destination)
INPUT  : Graph G with nodes and weighted edges, a source node and a destination
node
OUTPUT : the shortest route from source to destination and its total cost

1.  for each node v in G do
2.     dist[v] <- infinity
3.     prev[v] <- undefined
4.  end for
5.  dist[source] <- 0
6.  PQ <- priority queue that holds (distance, node), insert (0, source)
7.
8.  while PQ is not empty do
9.     u <- remove the node with the smallest distance from PQ
10.    if u is already visited then
11.           skip and continue the loop
12.    end if
13.    mark u as visited
14.    for each neighbour v of u do
15.           newDist <- dist[u] + weight(u, v)
16.           if newDist < dist[v] then
17.           dist[v] <- newDist
18.           prev[v] <- u
19.           insert (newDist, v) into PQ
20.           end if
21.    end for
22. end while
23.
24. path <- empty list
25. node <- destination
26. while node is defined do
27.    add node to the front of path
28.    node <- prev[node]
29. end while
30. return path and dist[destination]
```

5.4 Code

The pseudocode above is implemented using the Java programming language. The program stores the graph as an adjacency list and uses the built-in PriorityQueue class as the priority queue. It also measures the running time of the algorithm using System.nanoTime(), where the timer starts right before the main loop and stops right after it finishes. The full code is shown below.

```
import java.util.*;

public class AmbulanceRouting {

	// A small class to hold a node and its current distance for the priority
queue
	static class Node implements Comparable<Node> {
	int id;
	double dist;
	Node(int id, double dist) {
		this.id = id;
		this.dist = dist;
	}
	public int compareTo(Node other) {
		return Double.compare(this.dist, other.dist);
	}
	}

	// An edge that connects to a neighbour with a certain cost
	static class Edge {
	int to;
	double cost;
	Edge(int to, double cost) {
		this.to = to;
		this.cost = cost;
	}
	}

	public static void main(String[] args) {
	int n = 10; // number of nodes (A to J)
	String[] names = {"A", "B", "C", "D", "E", "F", "G", "H", "I", "J"};

	// Adjacency list for the undirected campus graph
	List<List<Edge>> graph = new ArrayList<>();
	for (int i = 0; i < n; i++) {
		graph.add(new ArrayList<>());
	}

	// Add every road as an undirected edge (both directions)
	addEdge(graph, 0, 2, 3.20); // A - C
	addEdge(graph, 0, 1, 2.50); // A - B
	addEdge(graph, 2, 4, 2.00); // C - E
	addEdge(graph, 2, 5, 0.70); // C - F
	addEdge(graph, 4, 7, 1.00); // E - H
```

```
        addEdge(graph, 5, 7, 2.00); // F - H
        addEdge(graph, 7, 8, 0.40); // H - I
        addEdge(graph, 7, 9, 0.65); // H - J
        addEdge(graph, 8, 6, 0.27); // I - G
        addEdge(graph, 8, 9, 0.35); // I - J
        addEdge(graph, 9, 6, 0.84); // J - G
        addEdge(graph, 6, 3, 0.90); // G - D
        addEdge(graph, 1, 3, 2.80); // B - D

        int source = 0;     // A = HSAAS
        int destination = 9; // J = KOSASS B

        double[] dist = new double[n];
        int[] prev = new int[n];
        boolean[] visited = new boolean[n];
        Arrays.fill(dist, Double.POSITIVE_INFINITY);
        Arrays.fill(prev, -1);

        // Start the timer right before the algorithm runs
        long startTime = System.nanoTime();

        dist[source] = 0.0;
        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.add(new Node(source, 0.0));

        while (!pq.isEmpty()) {
               Node current = pq.poll();
               int u = current.id;

               // Skip the node if it was already finalised before
               if (visited[u]) {
               continue;
               }
               visited[u] = true;

               // Relax every neighbour of u
               for (Edge e : graph.get(u)) {
               int v = e.to;
               double newDist = dist[u] + e.cost;
               if (newDist < dist[v]) {
                      dist[v] = newDist;
                      prev[v] = u;
                      pq.add(new Node(v, newDist));
               }
```

```
            }
        }

        // Stop the timer right after the algorithm finishes
        long endTime = System.nanoTime();
        long elapsed = endTime - startTime;

        // Print the shortest cost from the source to every node
        System.out.println("Shortest effective cost from A (HSAAS) to every
node:");
        for (int i = 0; i < n; i++) {
            System.out.printf("  A -> %s : %.2f%n", names[i], dist[i]);
        }

        // Rebuild the path from the source to the destination
        List<Integer> path = new ArrayList<>();
        for (int at = destination; at != -1; at = prev[at]) {
            path.add(at);
        }
        Collections.reverse(path);

        StringBuilder route = new StringBuilder();
        for (int i = 0; i < path.size(); i++) {
            route.append(names[path.get(i)]);
            if (i < path.size() - 1) {
            route.append(" -> ");
            }
        }

        System.out.println();
        System.out.println("Source node   : A (HSAAS)");
        System.out.println("Destination node : J (KOSASS B)");
        System.out.println("Optimal route : " + route.toString());
        System.out.printf("Total cost    : %.2f%n", dist[destination]);
        System.out.println();
        System.out.printf("Time taken    : %d nanoseconds%n", elapsed);
        System.out.printf("              :   %.4f   milliseconds%n",   elapsed   /
1_000_000.0);
        }

        // Helper that adds an edge in both directions
        static void addEdge(List<List<Edge>> graph, int u, int v, double cost) {
        graph.get(u).add(new Edge(v, cost));
        graph.get(v).add(new Edge(u, cost));
```

```
        }
}
```

5.5 Program Testing

The program was tested using the campus graph data from the model that was built earlier. The source was set to A (HSAAS) and the destination was set to J (KOSASS B). The console output of the program is shown below.

```
Shortest effective cost from A (HSAAS) to every node:
  A -> A : 0.00
  A -> B : 2.50
  A -> C : 3.20
  A -> D : 5.30
  A -> E : 5.20
  A -> F : 3.90
  A -> G : 6.20
  A -> H : 5.90
  A -> I : 6.30
  A -> J : 6.55

Source node   : A (HSAAS)
Destination node : J (KOSASS B)
Optimal route : A -> C -> F -> H -> J
Total cost    : 6.55

Time taken    : 417581 nanoseconds
              : 0.4176 milliseconds
```

From the output, the program first lists the shortest effective cost from A to every node. After that it prints the final route and the total cost. The result shows that the shortest route is A to C to F to H to J with a total cost of 6.55. If we change the letters back to the real names, this route is HSAAS to Pusat Sukan to Bukit Ekspo to KPZ to KOSASS B. Figure 5.3 shows the same route drawn on the campus graph.

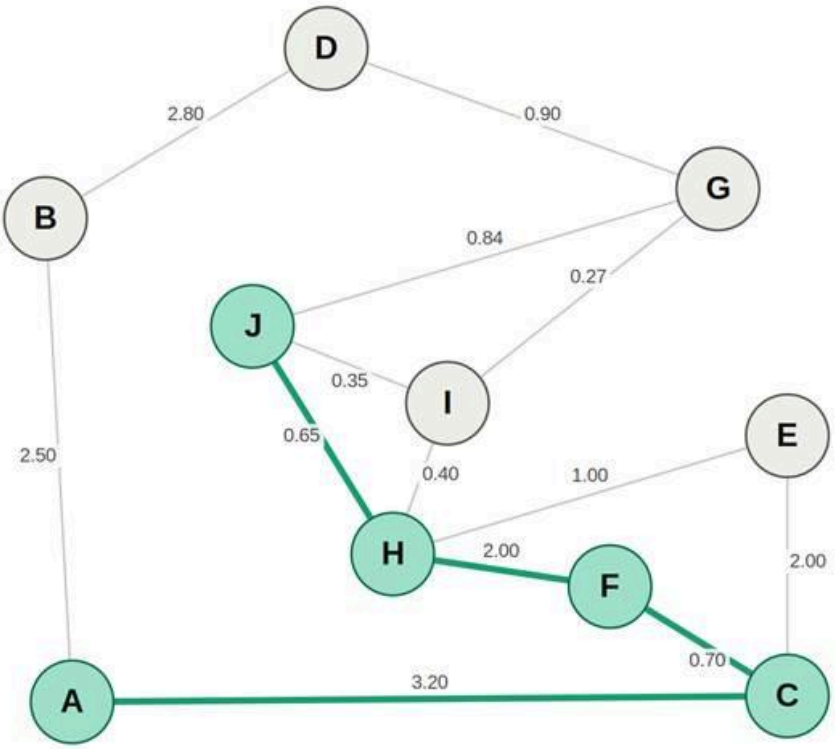


Figure 5.3 The shortest route from A to J highlighted on the campus graph

The output also shows the time taken by the algorithm, which is around 417,581 nanoseconds or about 0.42 milliseconds for this run. It is important to mention that this number is very small and it changes a little bit every time the program is run. This happens because our graph only has 10 nodes, so most of the measured time actually comes from the Java virtual machine warming up and creating the objects, and not from the pathfinding itself. The value is still useful as a simple proof that the algorithm runs very fast on this size of input, and it can be compared with the theoretical time complexity that is discussed later in Section 7.

6.0 Algorithm Verification

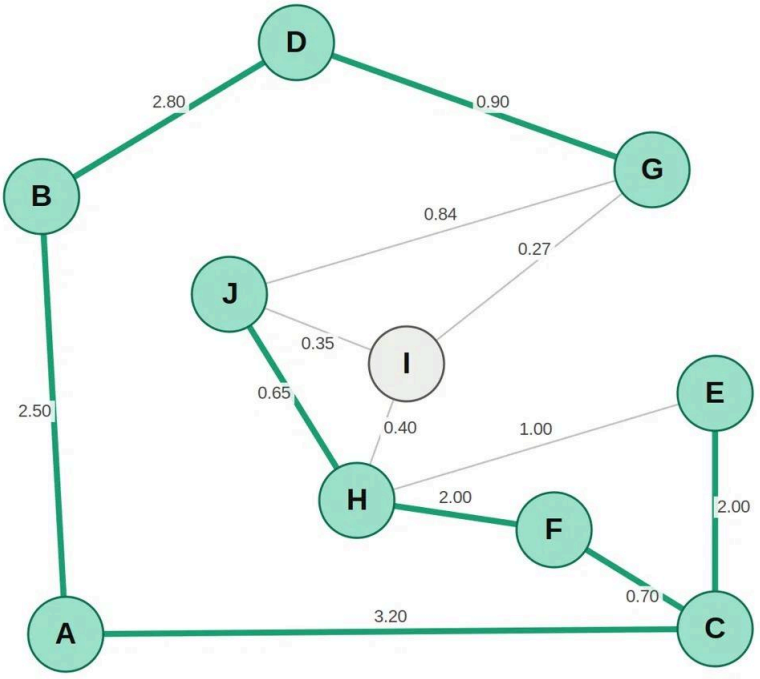
This section verifies that Dijkstra's algorithm is suitable and reliable for the Ambulance Routing System. The verification focuses on three important aspects: correctness verification, recurrence analysis, and asymptotic verification.

6.1 Correctness Verification

Dijkstra's Algorithm is a greedy shortest-path algorithm that selects the unvisited node with the smallest known distance from the source at each step. This greedy strategy ensures that once a node is chosen, its distance is the shortest possible and will not change.

The distance stored for each node is known as the tentative distance, which represents the shortest path currently discovered from the source. When a node is marked as visited, its distance becomes final. This property holds because Dijkstra's Algorithm assumes all edge weights, which represent travel costs such as distance or time, are non-negative. Since this condition is satisfied in the given graph, the correctness of the shortest path obtained is guaranteed.

To verify the correctness of the algorithm, the shortest path calculation is performed manually using the campus graph. The process is shown below.



Visited Nodes = {A, B, C, F, E, D, H, G, J}

Q:	A	B	C	D	E	F	G	H	I	J
	0									
A		2.50	3.20							
B			3.20	5.30						
C				5.30	5.20	3.90				
F				5.30	5.20			5.90		
E				5.30				5.90		
D							6.20	5.90		
H							6.20		6.30	6.55
G									6.30	6.55
J										6.55

Based on the table, the step-by-step execution of Dijkstra's Algorithm is as follows:

- First, we start at **A**. The next options are **B** with a tentative distance of 2.50 and **C** with a tentative distance of 3.20. Since **B** has the lowest tentative distance, **B** is selected.
- Next, from **B**, the available option is **D** (5.30), while **C** (3.20) remains unvisited. Since **C** has the lower tentative distance, the algorithm backtracks to **C** and selects it.
- Then, from **C**, the available options are **E** (5.20) and **F** (3.90), while **D** (5.30) remains unvisited. Since F has the lowest tentative distance, **F** is selected.
- After that, from F, the algorithm discovers **H** (5.90). Comparing all unvisited nodes, **E** (5.20) has the lowest tentative distance. Therefore, the algorithm backtracks to **E** and selects it.
- Subsequently, from **E**, another path to **H** is found with a tentative distance of 6.20. However, H already has a smaller tentative distance of 5.90, so no update is made. The algorithm can also backtrack to **D** (5.30), which has the lowest tentative distance among the remaining unvisited nodes. Therefore, **D** is selected.
- Next, from **D**, the algorithm discovers **G** (6.20). Comparing all unvisited nodes, **H** (5.90) has the lowest tentative distance. Therefore, the algorithm backtracks to H and selects it.
- Then, from **H**, the algorithm discovers **I** (6.30) and **J** (6.55). Among the remaining unvisited nodes, **G** (6.20) has the lowest tentative distance. Therefore, the algorithm backtracks to **G** and selects it.
- Furthermore, from **G**, the algorithm finds an alternative path to **I** with a tentative distance of 6.47. Since **I** already has a lower tentative distance of 6.30, no update is

made. Similarly, the alternative path to **J** has a tentative distance of 7.04, which is greater than the current value of 6.55, so no update is made. The algorithm then backtracks to **I** (6.30) and selects it.

- Finally, from **I**, the algorithm finds an alternative path to **J** with a tentative distance of 6.65. Since **J** already has a lower tentative distance of 6.55, no update is made. The algorithm then selects **J**.
- Since **J** is the destination node, the algorithm terminates. The shortest path obtained is **A** → **C** → **F** → **H** → **J** with a total cost of 6.55.

Therefore, the manual calculation produces the same result as the program output. This confirms that Dijkstra's Algorithm correctly identifies the optimal route from HSAAS to KOSASS B and guarantees the minimum travel cost for the ambulance routing problem.

6.2 Recurrence Analysis

The recurrence in Dijkstra's Algorithm occurs during the relaxation process, where the algorithm repeatedly checks whether a newly discovered path to a node is shorter than the current known path. The recurrence relation used is:

$$d[v] = min(d[v],\ d[u] + w(u, v))$$

Where:

- $d[v]$ is the current known shortest distance to node v.
- $d[u]$ is the shortest known distance to current node u.
- $w(u, v)$ is the weight of the edge between u and v.

This recurrence is applied whenever the algorithm explores a neighbouring node. If the newly calculated distance is smaller than the existing distance, the value is updated. Otherwise, the current value is retained.

The recurrence behaviour can be observed from the output of the Ambulance Routing System. One example occurs when the algorithm evaluates different routes to node H (KPZ).

- The first route to H is:
  A → **C** → **E** → **H**
  Total cost = 3.20 + 2.00 + 1.00 = 6.20
- Later, another route to H is discovered:

A → C → F → H

Total cost = 3.20 + 0.70 + 2.00 = 5.90

- According to the recurrence relation, the algorithm compares the existing distance of 6.20 with the newly discovered distance of 5.90.
- Since 5.90 is smaller, the distance of H is updated to 5.90.

Another example occurs when the algorithm evaluates different routes to the destination node J (KOSASS B).

- One route to J is:
  A → C → F → H → J
  Total cost = 5.90 + 0.65 = 6.55
- Another possible route is:
  A → C → F → H → I → J
  Total cost = 5.90 + 0.40 + 0.35 = 6.65
- The recurrence relation compares the current distance of 6.55 with the newly discovered distance of 6.65.
- Since 6.55 is smaller, the distance of J remains unchanged.

These examples demonstrate how the recurrence repeatedly compares existing distances with newly discovered distances and always keeps the smaller value. Through repeated relaxation, the distance estimates become increasingly accurate until the optimal values are obtained.

The examples shown above only illustrate a portion of the recurrence process. When the same recurrence relation is repeatedly applied to all nodes and edges in the graph, the resulting distance values are identical to those obtained in the manual verification in Section 6.1. Therefore, the recurrence analysis produces the same final output, which is the shortest route:

A → C → F → H → J

with a total cost of 6.55. This consistency between the recurrence process and the manual execution in Section 6.1 confirms that the algorithm correctly updates distance values and successfully determines the optimal ambulance route.

6.3 Asymptotic Verification

The asymptotic verification checks whether the behaviour of the implemented algorithm is consistent with its theoretical complexity. For Dijkstra's Algorithm implemented using an adjacency list and a min-priority queue, the time complexity is:

$$O((V + E) \log V)$$

where:

- V is the number of vertices.
- E is the number of edges.

For the Ambulance Routing System:

- Number of vertices, $V = 10$
- Number of edges, $E = 13$

Substituting these values into the complexity formula:

$= O((10 + 13) \log_2 10$

$\approx O(23 \times 3.32)$

$\approx O(76.4)$

This is a relatively small number of operations, which explains why the algorithm is able to quickly determine the shortest route from A (HSAAS) to J (KOSASS B). If the campus road network were expanded, the time complexity would scale as follows:

Graph Size (V nodes, E edges)	**Approximate Operations: $O((V + E) \log V)$**
$V = 10, E = 13$ (current)	~76
$V = 50, E = 80$	~693
$V = 100, E = 200$	~1,993
$V = 1000, E = 3000$	~39,900

Based on the table, the number of operations increases gradually as the graph becomes larger. This growth is significantly slower than algorithms such as Floyd-Warshall, which have a time complexity of $O(V^3)$. For example, when $V = 1000$, Floyd-Warshall would require approximately 1,000,000,000 operations, whereas Dijkstra's Algorithm requires only about 39,900 operations.

Therefore, the successful generation of the route **A** → **C** → **F** → **H** → **J** with a total cost of

6.55, together with the expected growth pattern shown in the table, confirms that the implementation behaves consistently with the theoretical asymptotic complexity of Dijkstra's Algorithm. This demonstrates that the algorithm is both correct and efficient for the ambulance routing problem.

7.0 Analysis of the Algorithm

7.1 Time Complexity

Depending on how the graph is stored and what data structure is used for the priority queue, the time complexity of Dijkstra's algorithm will have varying results. In this implementation, the graph is stored as an adjacency list and a min-heap based priority queue (Java's built-in PriorityQueue) is used. There are three cases that should be considered:

Best Case: $O((V + E) \log V)$

The best case occurs when the graph is sparse, meaning the number of edges is close to the number of vertices. In this scenario, since there are fewer neighbours to relax at each step, priority queue operations are minimal. Typically, the average-case time complexity of Dijkstra's algorithm is $O((V + E) \log V)$. Real-world graphs are often neither extremely sparse nor fully connected, which is why the algorithm performs well in real-world environments (GeeksforGeeks, 2024). For the UPM campus graph with $V = 10$ and $E = 13$ this is the effective case since the graph is small and sparse.

Average Case: $O((V + E) \log V)$

Just like the best case, the average case has the same complexity when using a priority queue with an adjacency list representation. Each vertex is processed once, and each edge may trigger at most one priority queue update, giving a time complexity of $O((V + E) \log V)$ when using an adjacency list together with a min-priority queue (MIT OpenCourseWare, 2020). This holds across most typical graph structures encountered in real-world applications.

Worst Case: $O(V^2 \log V)$

The worst case occurs when the graph is very dense, meaning almost every node is connected to every other node. In the worst-case scenario. When the graph is dense with many edges, it makes priority queue operations less efficient. The time complexity in this case is $O(V^2 \log V)$ (GeeksforGeeks, 2024). In this scenario, the algorithm performs many more relaxation operations and priority queue insertions because almost every node has many neighbours. However, the worst does not apply to the UPM campus road network because the graph is sparse.

7.2 Space Complexity

Space complexity refers to the amount of memory needed to run Dijkstra's algorithm. The following data structures are used in this implementation:

a) **Adjacency list:** Stores all nodes and their connected edges. This requires $O(V + E)$ space where V is the number of nodes and E is the number of edges.

b) **Distance array (dist[]):** Stores the current shortest known distance to every node. This takes $O(V)$ space.

c) **Previous node array (prev[]):** Stores the predecessor of each node to allow path reconstruction. This takes $O(V)$ space.

d) **Visited array (visited[]):** Tracks whether each node has been finalised. This takes $O(V)$ space.

e) **Priority queue:** In the worst case, the priority queue can hold up to $O(E)$ entries since each edge can potentially add one entry to the queue.

f) **Total space complexity:** $O(V + E)$

The space complexity of Dijkstra's algorithm is $O(V + E)$, which accounts for the graph representation, the distance array, and the priority queue storage (GeeksforGeeks, 2024).

In this scenario with $V = 10$ and $E = 13$:

$$O(V + E) = O(10 + 13) = O(23) \text{ units of memory}$$

This is a very small memory footprint. This algorithm would run without any memory issues even on a basic computer with limited resources. It will remain manageable, assuming the graphs were scaled up to cover the entire UPM campus with hundreds of buildings and roads, the space usage would still grow linearly with the size of the graph.

Table 7.1: Space complexity breakdown of Dijkstra's algorithm

Data Structures	**Spaced Used**
Adjacency list	$O(V + E)$
Distance Array	$O(V)$
Previous Array	$O(V)$
Visited Array	$O(V)$
Priority Queue	$O(E)$ at most
Total	$O(V + E)$

8.0 Conclusion

In conclusion, by using Dijkstra's Algorithm an Ambulance Routing System for Universiti Putra Malaysia (UPM) has been successfully developed to determine the optimal route between Hospital Sultan Abdul Aziz Shah (HSAAS) and KOSASS B during an emergency situation. The campus road network was modelled as a weighted graph, where each location was represented as a node and each road was represented as an edge with an effective travel cost that considered both distance and traffic congestion.

After evaluating multiple different shortest-path algorithms, including Breadth-First Search (BFS), Floyd-Warshall, and Greedy Best-First Search, Dijkstra's Algorithm was selected because it guarantees the optimal shortest path for graphs with non-negative edge weights while maintaining good computational efficiency. The implementation using an adjacency list and a min-priority queue successfully identified the optimal route from HSAAS to KOSASS B with a total effective travel cost of 6.55.

Verified through manual calculations, recurrence analysis, and asymptotic verification, the correctness of the algorithm produced consistent results. Furthermore, the analysis showed that the algorithm has a time complexity of $O((V + E) \log V)$ and a space complexity of $O(V + E)$. This makes the algorithm suitable for both the current campus road network and larger graph structures.

Overall, for emergency route planning the project demonstrates that Dijkstra's Algorithm is a reliable, accurate, and efficient solution. The system can help emergency vehicles by considering both road distances and traffic conditions, so that the emergency vehicle can reach their destinations more quickly, potentially improving response times and reducing risks during critical situations.

References

Abiy, T., Pang, H., Williams, C., Khim, J., & Ross, E. (n.d.). *Dijkstra's shortest path algorithm*. Brilliant. https://brilliant.org/wiki/dijkstras-short-path-finder/

Carnegie Mellon University. (2015). *Chapter 16: Shortest paths*. https://www.cs.cmu.edu/afs/cs/academic/class/15210-s15/www/lectures/shortest-paths-notes.pdf

Chauhan, P. (2024, December 13). *Understanding Dijkstra's algorithm: From theory to implementation*. DEV Community. https://dev.to/frorning/understanding-dijkstras-algorithm-from-theory-to-implementation-1ed

Code and Debug. (2025). *Dijkstra's algorithm with a priority queue*. https://codeanddebug.in/blog/dijkstra-algorithm-with-a-priority-queue/

Code Intuition. (n.d.). *Dijkstra algorithm explained: Why it actually works*. https://www.codeintuition.io/blogs/dijkstra-algorithm-explained

Cormen, T. H. (2018). *Dijkstra's algorithm* [Lecture notes, CS 10]. Dartmouth College. https://www.cs.dartmouth.edu/~thc/cs10/lectures/0509/0509.html

GeeksforGeeks. (2024). *Time and space complexity of Dijkstra's algorithm*. https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-dijkstras-algorithm/

GeeksforGeeks. (2025). *Greedy best first search algorithm*. https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/

Gerdelan, A. (n.d.). *Finding the shortest path: Greedy best-first search* [Lecture slides]. Trinity College Dublin. https://antongerdelan.net/teaching/cs3d05a/Lecture%2024%20-%20Greedy%20Best-First%20Search.pdf

Hernandez, Y. (2020, September 28). *Dijkstra's shortest path algorithm: A visual introduction*. freeCodeCamp. https://www.freecodecamp.org/news/dijkstras-shortest-path-algorithm-visual-introduction/

Malhotra S. (2018, July 12). *Exploring the applications and limits of breadth-first search to the shortest paths in a weighted graph*. https://www.freecodecamp.org/news/exploring-the-applications-and-limits-of-breadth-first-search-to-the-shortest-paths-in-a-weighted-1e7b28b3307/

MIT OpenCourseWare. (2020). *Introduction to algorithms (6.006), Recitation 13: Dijkstra's*

algorithm. Massachusetts Institute of Technology. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

MIT OpenCourseWare. (2020). *Lecture 13: Dijkstra's algorithm* [Lecture notes, 6.006]. Massachusetts Institute of Technology. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/d819e7f4568aced8d5b59e03db6c7b67_MIT6_006S20_lec13.pdf

Mount, D. (2025). *Lecture 4: Graph shortest paths – Dijkstra and Bellman-Ford*. University of Maryland. https://www.cs.umd.edu/class/spring2025/cmsc451-0101/Lects/lect04-graph-shortest-path.pdf

Patel, A. (n.d.). *Introduction to A*: Comparing search algorithms*. Stanford Theory Group. http://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html

Scaler Topics. (2022). *Dijkstra's algorithm*. https://www.scaler.com/topics/data-structures/dijkstra-algorithm/

Trital, P. (2025, November 28). *Dijkstra's algorithm*. Medium. https://medium.com/@prajun_t/dijkstras-algorithm-fc4a425c2b56

Wang, Y., et al. (2018). *The comparison of three algorithms in shortest path issue*. *IOP Conference Series: Journal of Physics, 1087*(2), 022011. https://iopscience.iop.org/article/10.1088/1742-6596/1087/2/022011/pdf